

AI-Powered Chatbot Platform for University Information Services

Apisit Yambangyang
Computer Engineering
Mae Fah Luang University
Chiang Rai, Thailand
apisityambangyang@gmail.com

Peerawit Kubang
Computer Engineering
Mae Fah Luang University
Chiang Rai, Thailand
tuntun.peerawit@gmail.com

Peerapong Wongwichai
Computer Engineering
Mae Fah Luang University
Chiang Rai, Thailand
bunzzbranzz@gmail.com

Mahamah Sebakor
Center for Information Technology Services
Mae Fah Luang University
Chiang Rai, Thailand
mahamah.seb@mfu.ac.th

Abstract—AI-Powered Chatbot Platform for University Information Services is a simple web application that aims to provide information about the university. The chatbot developed is a web-based application that leverages AWS Bedrock models (Claude 3.5) for conversational AI to communicate with users. ChatGPT and Gemini are examples of online applications that have been developed. This campus information chatbot is designed to have friendly conversations and respond to user-related and university-related questions, such as sports, grades, or recommending courses for the next semester. It uses models from AWS Bedrock to power the chatbot conversation. This project focuses on building a chatbot that allows students to conveniently ask questions through the university's website. A chatbot is a program that can hold real conversations through text. With AI, chatbots can simulate human-like interactions. While humans usually respond based on mood and emotions, chatbots follow fixed rules, making them consistently polite and accurate. Students can ask the chatbot questions at any time and get fast responses. The chatbot can handle thousands of users at the same time and operate 24/7, which greatly improves work efficiency.

Index Term — Artificial Intelligence, Chatbot, GUI

I. INTRODUCTION

In recent years, many universities have started using chatbots to improve the way they deliver information and services to students. These systems allow students to access details such as grades, extracurricular activities, and even personalized course suggestions in a more natural and convenient way [7], [8].

The shift from older Natural Language Processing pipelines toward foundation models has changed how chatbots are built. Instead of relying on fixed keyword-based rules, institutions can now integrate advanced Large Language Models that provide more flexible and adaptive responses. Cloud platforms such as AWS Bedrock make this easier by offering direct access to powerful models without requiring the university to manage its own training or infrastructure [14], [17]. Among these, Anthropic Claude 3.5 stands out because it

supports large contexts, better reasoning, and safe deployment in academic environments [16].

By using Claude 3.5 through AWS Bedrock, the chatbot can handle real-time, multi-turn conversations and generate recommendations that are tailored to each student. When combined with secure authentication methods like SAML or JWT [1], [2], efficient memory management using Redis and ChromaDB [5], and fast communication through WebSocket [3], [4], the system becomes both scalable and reliable.

This research introduces a chatbot platform designed specifically for universities. The system is powered by AWS Bedrock (Claude 3.5) and includes authentication, tool integration, and memory components. Together, these allow the chatbot to deliver safe, context-aware, and continuously improving interactions between students and university services [6], [15].

II. LITERATURE REVIEW

A. Rule-based and Basic Chatbot Systems

The earliest university chatbots mostly used rule-based methods or AIML. They worked by matching certain keywords to pre-defined answers, usually for FAQ-style questions [9]. These systems were easy to build and deploy but could not handle complex queries or multi-turn conversations.

Drawbacks: They lacked flexibility, often gave repetitive responses, and were unable to provide personalized support.

B. LLM-based Chatbots and Foundation Models

The development of Large Language Models (LLMs) has moved educational chatbots far beyond keyword matching. Instead of manually creating NLP pipelines, institutions can now use foundation models to understand queries, maintain context, and respond with more natural language [11], [12]. AWS Bedrock provides access to

enterprise-ready models such as Claude 3.5, which is known for strong reasoning abilities and safer deployment in real environments [14], [16], [17].

Drawbacks: Despite their benefits, LLMs can sometimes produce inaccurate information, contain bias, or raise privacy issues if not carefully managed [10].

C. Infrastructure: Security, Authentication, and Real-time Communication

For a university chatbot to run smoothly, the background systems are just as important as the chatbot itself. Secure login is needed so that only students and staff can use it, which is why methods like SAML and JWT are often applied [1], [2]. To make conversations fast and continuous, WebSocket is used for real-time data transfer [3], [4]. On top of that, Redis stores recent messages so the chatbot remembers the flow of a conversation, while ChromaDB keeps older information for future use [5].

Drawbacks: While this setup makes the chatbot more useful, it also adds layers of complexity. Keeping everything reliable means constant checks and updates, which can be a challenge for university IT teams [15].

D. User Engagement and Ethical Considerations

One clear benefit of AI chatbots is how they help students anytime, even outside office hours. This reduces staff workload and keeps students more engaged [7], [8], [12]. But at the same time, depending too much on models like Claude 3.5 can bring risks. These include biased answers, misinformation, or students becoming overly reliant on the system [10], [16]. To build trust, transparency and human oversight are still necessary.

Drawbacks: While efficiency improves, universities need to balance AI with real human support so that fairness and reliability are not lost.

III. Proposed System

The system we propose is built as a chatbot platform that focuses on university services. At the center of the design is AWS Bedrock with the Claude 3.5 model, which takes care of understanding questions and generating answers. Instead of building a traditional NLP pipeline, the chatbot sends requests directly to Bedrock through its Runtime API [14], [17].

Students use the chatbot through the university website. They can ask about things like course registration, program details, or activities. Answers are sent back immediately using WebSocket [2], [3], so the conversation feels natural and flows smoothly.

To make the chatbot more useful, a set of tools is connected to it [6]:

- *web_search* – Performs online searches with DuckDuckGo (with Google fallback)
- *calculator* – Handles mathematical calculations and expressions.
- *current_date* – Provides date and time functions to give context-aware answers.

- *memory_tools* – Searches and embeds previous chat history so the bot can “remember” past conversations.
- *retrieval_tools* – Connects to knowledge bases for contextual responses [13].

On top of this, the system can work with uploaded documents or trusted web pages. These sources are broken down into small sections, embedded through Bedrock [14], and saved in ChromaDB [5]. Later, the chatbot can use them to give more specific answers.

In short, the design combines three ideas: conversational AI, memory management, and flexible knowledge integration. This combination helps the chatbot improve over time, while still keeping the system secure and reliable [15], [17].

IV. DESIGN OF DINDIN AI

DinDinAI, is Displayed in Fig.1. The graphical user interface (GUI) is important in chatbots because it improves the user experience. The chatbot’s front end is then created using TypeScript, CSS, HTML. For the back end, I use TypeScript and Node.js in the runtime environment.

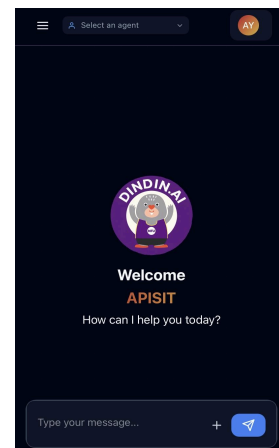


Fig. 1. Design of ChatAI

V. IMPLEMENTATION PROCESS

A. WebSocket Connection and Session Handling

The first step in building the system was to set up a WebSocket channel for real-time communication [2], [3]. We chose WebSocket over regular HTTP because it made the interaction feel smoother; users did not have to wait for a full request–response cycle each time. To keep the connection secure, JSON Web Tokens (JWT) were used during the handshake [1]. If the token was valid, the server created a session record containing details such as user ID, the room joined, and the latest activity [4]. This made it easier to manage multiple rooms, though in early

testing we sometimes had problems when users lost their connection and the session data was not cleared correctly. That issue had to be fixed by adding timeout checks.

B. Message Processing Flow

Once connected, user input went through a simple but structured pipeline. Messages were first logged and checked, and then passed to the chat service. Depending on the room or the type of question, the system applied a different configuration. For example, academic questions were routed through an agent that could connect to a knowledge source, while casual ones were handled with a web search [6]. The LangChain framework helped manage these tools without us needing to write custom routing logic for each case. In practice, this saved time, but we also noticed that adding too many tools at once made the pipeline slower, so some trade-offs had to be made.

C. Memory Management

Conversation flow depended heavily on memory. For short-term memory, Redis stored only the latest few messages, which was enough to maintain context within an active session [5]. This part worked well, especially under load, since Redis could retrieve data instantly. Long-term context was more difficult. We used ChromaDB to store older messages as embeddings along with metadata like topic tags and timestamps [5]. Every tenth message was embedded and added to the database. In testing, this worked for bringing back older topics, but it also produced some irrelevant matches when the embeddings overlapped too much. We had to adjust the indexing method to reduce this noise.

D. Integration with AWS Services

The final layer was integration with AWS. Bedrock provided the language model through its Runtime API [14], [16], [17]. This was convenient since we did not need to host or train a large model ourselves. The agent generated prompts, sent them to Bedrock, and streamed the output back to the user. Streaming turned out to be more engaging, since users could see the reply forming instead of waiting for a block of text. We also connected Amazon S3 for archiving logs and embeddings [15]. This became useful later, because Redis and ChromaDB were only meant to handle active data, and S3 gave us a reliable backup when we needed to review old sessions.

VI. RESULTS

In Fig. 2. DinDinAi integrates LangChain and AWS Bedrock to provide real-time chat support services to Mae Fah Luang University. Authentication works through two methods: student and staff login via MFU SSO using university email addresses, while administrator authentication uses JWT authentication. WebSocket enables real-time communication with the AWS Bedrock underlying model. LangChain integration enables Redis

memory management and operation of the Tool Registry. The Tool Registry provides web search and vector search functionality via a knowledge base. The knowledge base allows users to create customized document repositories, enabling DinDinAi to better meet the specific needs of the department.

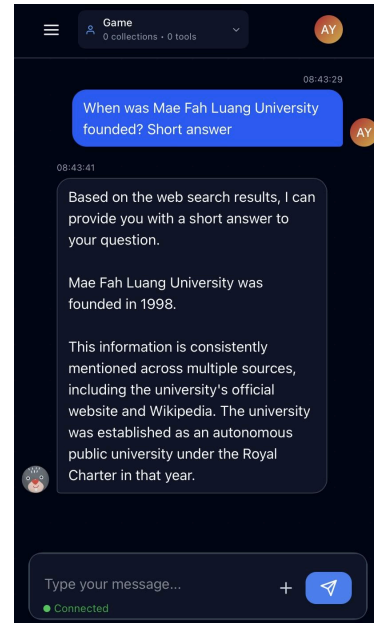


Fig. 2. ScreenShot chat of ChatAI

VII. CONCLUSION

Working on this project showed us how an AI chatbot can actually make university services more accessible. We started with the idea of creating a simple tool for answering questions, but along the way we realized the system could also reduce routine tasks for staff and give students faster support.

Of course, there are still limitations. Privacy, accuracy, and long-term maintenance are things we could not fully solve in this version. These challenges remind us that technology alone is not enough—it needs careful management and human oversight.

Overall, the project was a good step toward combining AI with everyday university life. The chatbot does not replace people, but it helps them. More importantly, it gave us the chance to see how technical work can turn into something practical and useful for a wider community.

REFERENCES

- [1] A. E. M. Obinna et al., "A JSON Token-Based Authentication and Access Management Schema for Cloud SaaS Applications," arXiv, Oct. 23, 2017.
- [2] A. Demir and M. Korkmaz, "Real-Time Web Applications with Node.js: Leveraging WebSockets and Socket.IO for Seamless Communication," Int. J. Trend Sci. Res. Dev., vol. 4, no. 5, pp. 1766–1773, Aug. 2020.

- [3] Bhaskar Pandey and Pramod Kumar, "Security Risks and Best Practices for Securing WebSocket Communications," *Int. J. Novel Res. Dev.*, vol. 9, no. 7, Jul. 2024.
- [4] Z. Miller et al., "Flexible Session Management in a Distributed Environment," *arXiv*, Nov. 2, 2010.
- [5] S. Toro et al., "Dynamic Retrieval Augmented Generation of Ontologies using Artificial Intelligence (DRAGON-AI)," *J. Biomed. Semantics*, Jun. 12, 2024.
- [6] "LangChain and Agents," in *Large Language Models Projects*, P. Martra (ed.), Springer, September. 19, 2024, pp. 63–132.
- [7] Davar, N. F., Dewan, M. A. A., & Zhang, X., "AI Chatbots in Education: Challenges and Opportunities," *Information*, vol. 16, no. 3, Art. 235, 2025.
- [8] Hidayat-ur-Rehman, I., "Examining AI competence, chatbot use and perceived autonomy as drivers of students' engagement in informal digital learning," *Journal of Research in Innovative Teaching & Learning*, vol. 17, no. 2, pp. 196–212, 2024.
- [9] Hasan Abu-Rasheed et al., "Supporting Student Decisions on Learning Recommendations: An LLM-Based Chatbot with Knowledge Graph Contextualization for Conversational Explainability and Mentoring," *arXiv preprint*, Jan. 2024.
- [10] Ramteja Sajja et al., "Evaluating AI-Powered Learning Assistants in Engineering Higher Education: Student Engagement, Ethical Challenges, and Policy Implications," *arXiv preprint*, Jun. 2025.
- [11] Thanarat Kingchang, Pinanta Chatwattana, and Panita Wannapiroon, "Artificial Intelligence Chatbot Platform: AI Chatbot Platform for Educational Recommendations in Higher Education," *International Journal of Information and Education Technology*, vol. 14, no. 1, pp. 34–41, Jan. 2024.
- [12] Gustavo Navas-Reascos et al., "Exploring the Effectiveness of Advanced Chatbots in Educational Settings: A Mixed-Methods Study in Statistics," *Applied Sciences*, vol. 14, no. 19, Art. 8984, Oct. 2024.
- [13] "Retrieval-Augmented Generation (RAG) Applications with AutoGen," Microsoft AutoGen blog, Oct. 2023.
- [14] Amazon Web Services, "Build generative AI solutions with Amazon Bedrock," AWS Blog, 13 Jun. 2025.
- [15] Milvus, "What are the advantages of using Amazon Bedrock for companies that are already heavily using AWS services," AI Quick Reference, 5 Feb. 2025.
- [16] Anthropic, "Introducing Claude 3.5," Anthropic Blog, 2025.
- [17] Amazon Web Services, "Amazon Bedrock Documentation," AWS Docs, 2025.